What Makes a Virtual ISA Programmable by AI Agents? A Controlled Study of Agent-Facing Interfaces to StableHLO

{Author Name(s)}
{Affiliation(s)}
{email}

***Abstract*—AI agents increasingly write code at the virtual-ISA layer, yet benchmarks score the agent, not the *interface* the ISA exposes to it. We fix one IR (StableHLO), one pinned verifier and executor, the tasks and the agent loop, and vary only how semantic documentation reaches the agent: none, syntax only, natural-language prose, a structured JSON contract with the same canonical facts and the same code examples at matched length, and on-demand retrieval through a tool (free, forced, or with the cross-cutting rules given upfront). A pre-registered study on 267 held-out tasks (114 synthetic, 153 lowered from KernelBench PyTorch modules) with independent oracles and three GPT-4.1 snapshots (5,005 trials) finds that: (1) documentation is the dominant lever ($+16$ to $+33$ pp first-attempt correctness over none); (2) once representation confounds are removed, prose beats the structured contract for every model (-6.0, -7.9, -12.1 pp), most strongly on real-world tasks, overturning the opposite trend in our own earlier unmatched comparison; and (3) on-demand access underperforms upfront delivery by 4–32 pp, because the strongest model never calls the tool unless forced and partial upfront documentation suppresses lookup. Over 99% of failures are rejected statically: agent-facing VISA design is first a problem of making well-formedness learnable. Harness, tasks, protocol and logs are released.**

I. Introduction

A virtual ISA decouples programs from rapidly changing hardware: front ends target the virtual layer and back ends absorb the churn [1], [2]. Increasingly the producer at that layer is an AI agent that writes, repairs and lowers code. This makes the presentation of the ISA to a non-human programmer a design dimension in its own right. Benchmarks score models on GPU kernels [3]–[5], LLVM IR optimization [6] or unfamiliar accelerator ISAs [7]; they answer "how good is the agent?" for a given interface. We ask the complementary question: *for a fixed agent and a fixed IR, which properties of the agent-facing interface change whether the agent writes correct programs?*

We study StableHLO [8], a versioned, specification-backed MLIR dialect [2] that serves as a portability layer between ML frameworks and compilers. The manipulation is controlled at the level of individual facts: a canonical inventory anchored to the pinned specification is rendered as prose, as a formal contract, or behind a retrieval tool.

Contributions. (i) A controlled methodology and harness that vary agent-facing ISA documentation while holding IR, toolchain, tasks and agent loop fixed, with an automated equivalence audit. (ii) A 328-task StableHLO suite: synthetic tasks plus 176 KernelBench modules lowered and validated end-to-end against PyTorch oracles. (iii) Two studies: an exploratory one that exposed confounds in our own treatments, and a pre-registered one (5,005 trials) that removes them. (iv) Design implications for agent-facing virtual ISAs, including results that reverse the ones we obtained before matching the treatments.

II. Related Work

KernelBench [3] and TritonBench [4] evaluate LLM generation of efficient GPU kernels from PyTorch-level references. TritonGym [5] adds agentic tool use to separate model ability from workflow design. CIRBench [6] evaluates LLMs on analyzing, repairing and optimizing LLVM IR, and KernelCraft [7] benchmarks agents writing kernels for emerging accelerator ISAs. SWE-agent [9] shows that the design of the *agent-computer interface* changes agent success, and DocPrompting [10] shows that retrieved documentation helps code generation for unseen APIs. We bring that interface view to the virtual ISA. We hold model, IR and tasks fixed, and manipulate the representation and access mode of the ISA's semantic documentation at the level of fact-equivalent treatments.

III. Study Design

We ran two studies. **Study 1** (exploratory; 114 synthetic tasks, 2,052 trials) surfaced two confounds in our own treatments and an unexpected uptake failure in on-demand documentation. **Study 2**, the study reported in the rest of this paper, was pre-registered after Study 1. It fixes those confounds, adds real-world tasks derived from KernelBench, and promotes the Study 1 post-hoc conditions to planned ones. Protocol, hashes and all trial logs are released.

A. Fixed IR, toolchain and agent loop

Programs are StableHLO modules with a single `@main`. We parse and verify them with the MLIR/StableHLO bindings

TABLE I
INTERFACE TREATMENTS (STUDY 2). TOK.: MEAN DOCUMENTATION TOKENS (O200K) OVER 328 TASKS.

	Condition	Facts upfront	Rendering	Tok.
–	NONE	none	–	0
A	MINIMAL	syntax	prose	951
N	SYNTAX-ONLY STR.	syntax	JSON	850
B	PROSE	all	prose	1455
C	STRUCTURED	all	JSON + IDs	1469
D	INTERACTIVE	none (on demand)	JSON, tool	0
D^*	INT. FORCED	as D, 1st call forced	JSON, tool	0
D^+	INT. CORE	module; rest on demand	JSON, tool	362

in jaxlib 0.11.2 (StableHLO 1.20.0) and execute them on XLA:CPU in a sandboxed subprocess with a 60 s timeout and an operation allow-list: StableHLO plus three CHLO element-wise ops, with no `custom_call`. The task fixes the signature exactly. The agent receives a system prompt, the documentation (if any) and the task, and replies with a program. Failures are returned verbatim (verifier diagnostic, or the first mismatching oracle element), with up to two repairs. Documentation stays in context, and in the interactive conditions the retrieval tool remains available (6 calls). Each trial is a fresh conversation at temperature 0, one per task and condition.

B. Canonical facts and interface treatments

The inventory has 96 facts over 23 operation groups (25 syntax, 16 type, 32 shape, 23 semantics), each anchored to a clause of the pinned specification [8] and authored twice: as a natural-language sentence and as a formal predicate in spec notation. *Study 2 fix 1:* every syntax fact carries the same general schema and the same concrete instance in both renderings (e.g. both `array<i64: ...>` and `array<i64: 1>`). In Study 1, prose carried only the instance and the formal rendering only the schema. *Fix 2:* the structured contract is compact JSON, so its mean length is $0.98\times$ that of prose over all 328 tasks (Study 1: $1.31\times$). An automated audit checks every task: identical fact-ID sequences, each rendering present verbatim, and no IDs leaking into prose. All 29 code examples verify with the pinned toolchain. A task's fact set is the cross-cutting `module` facts (program structure, types, the integer-list attribute encoding, no implicit broadcasting) plus the facts of the operation groups the task lists as relevant. Every condition receives the identical task text, signature and list of relevant groups.

C. Tasks and independent oracles

Synthetic (114 tasks). 38 parametric templates over broadcasting, reductions, contractions, layout, slicing/padding, masking/indexing, conversion and composites (e.g. softmax), each instantiated three times with distinct dimension sizes; an additional instance per template is a development split. Each task is specified mathematically, and its oracle is a numpy reference evaluated on three held-out cases that include semantic traps (all-negative rows for max-reductions, exact zeros for strict comparisons, out-of-range `dynamic_slice` starts).

KernelBench-derived (153 tasks). We converted all 200 level-1 and level-2 tasks of KernelBench [3] (MIT licence): single operators, normalizations, pooling, convolutions, losses, and fused operator chains. The pipeline (i) shrinks integer size literals deterministically, preserving divisibility when required; (ii) exposes all module parameters as explicit `@main` arguments, keeping training-mode semantics as KernelBench evaluates them; (iii) uses PyTorch eager outputs on three cases, each with freshly initialised parameters and inputs, as the oracle (rtol$=10^{-3}$); and (iv) produces a reference StableHLO program via `torch.export` and `torchax`. The reference is never shown to agents; it only demonstrates that the task is expressible and that the oracle and toolchain agree. The agent sees the PyTorch module source, the argument mapping and the signature. We excluded 24 tasks, each with a logged reason: 18 were too large even after shrinking, 2 had shape couplings broken by shrinking, 2 use dropout, 1 takes a non-tensor input, and 1 failed validation because its reference disagreed with PyTorch. Of the remaining 176, a 10% hash split (23 tasks) forms the development set, leaving 153 evaluation tasks; the full evaluation population is 267 tasks.

Validation (gate G1). All 328 reference programs (152 synthetic, 176 KernelBench) parse, verify, execute and match their oracles. On synthetic tasks the oracle rejected all 132 non-equivalent executing mutants of the reference solutions; the 13 survivors are semantically equivalent.

D. Models, budget and statistics

We used three dated snapshots of one family, `gpt-4.1-nano/-mini/-2025-04-14`. A local Qwen2.5-7B solved 0 of 12 development trials and was not used. The total API budget was capped at $35 and enforced by the runner. Trials were ordered task-blocked (tasks shuffled, conditions shuffled within each task), so an outcome-blind budget stop leaves complete, paired tasks. The primary endpoint is first-attempt semantic correctness, and the primary contrast is C−B. Eight paired contrasts were pre-specified (Table III). For each model we report the mean paired difference with a 95% cluster-bootstrap CI (10k resamples; clusters are templates for synthetic tasks and individual tasks for KernelBench) and exact McNemar tests, Holm-corrected within each model. Per-suite results are secondary, and the failure taxonomy (assembly syntax, attribute encoding, literal, shape/type, unknown op, wrong values) was fixed before the runs.

IV. RESULTS

All pre-registered trials completed for gpt-4.1-nano and -mini (267 tasks each, 2,136 trials per model). The gpt-4.1 run reached its share of the budget after 733 trials; under the frozen task-blocked rule it is analysed on its 91 complete tasks (39 synthetic, 52 KernelBench). The total cost was $34.52.

TABLE II
SEMANTIC CORRECTNESS (%) OVER ALL EVALUATION TASKS: FIRST ATTEMPT (1ST) AND AFTER $\leq$2 REPAIRS (REP.). n = 267 TASKS FOR NANO AND MINI, 91 FOR GPT-4.1.

	4.1-nano		4.1-mini		4.1	
Condition	1st	rep.	1st	rep.	1st	rep.
None	2.6	3.4	0.4	0.7	14.3	19.8
Minimal (A)	18.4	19.9	33.0	40.4	41.8	48.4
Syntax-only struct. (N)	15.0	17.2	32.6	40.8	39.6	47.3
Prose (B)	22.8	24.7	41.2	48.7	56.0	60.4
Structured (C)	16.9	23.2	33.3	39.7	44.0	53.8
Interactive (D)	12.7	13.5	19.9	21.7	12.1	16.5
Int. forced (D*)	14.2	15.7	22.1	23.6	29.7	31.9
Int. core (D+)	3.7	4.5	20.6	34.8	29.7	37.4

TABLE III
PRE-REGISTERED PAIRED CONTRASTS, FIRST ATTEMPT, ALL TASKS. Δ: SECOND MINUS FIRST CONDITION IN EACH PAIR, IN PP; 95% CLUSTER-BOOTSTRAP CI; DISC. = TASKS SOLVED ONLY BY THE FIRST / ONLY BY THE SECOND CONDITION; HOLM WITHIN MODEL.

Model	Contrast	Δ (pp)	95% CI	disc.	p_{Holm}
4.1-nano	Structured – Prose	-6.0	[-12.5, +0.0]	12/28	0.083
	Interactive – Structured	-4.1	[-9.0, +0.7]	11/22	0.240
	Int. forced – Interactive	+1.5	[-0.8, +4.1]	9/5	0.424
	Int. core – Interactive	-9.0	[-14.7, -3.8]	3/27	<0.001
	Int. core – Structured	-13.1	[-19.8, -6.6]	5/40	<0.001
	Prose – Minimal	+4.5	[+0.0, +9.1]	18/6	0.091
	Minimal – None	+15.7	[+10.1, +21.8]	43/1	<0.001
	Syntax-only struct. – Minimal	-3.4	[-8.6, +1.5]	11/20	0.299
4.1-mini	Structured – Prose	-7.9	[-12.6, -3.7]	4/25	<0.001
	Interactive – Structured	-13.5	[-19.3, -8.0]	4/40	<0.001
	Int. forced – Interactive	+2.2	[-1.1, +5.5]	14/8	0.859
	Int. core – Interactive	+0.7	[-5.8, +7.0]	22/20	1.000
	Int. core – Structured	-12.7	[-18.6, -7.2]	4/38	<0.001
	Prose – Minimal	+8.2	[+3.7, +13.3]	26/4	<0.001
	Minimal – None	+32.6	[+24.3, +40.5]	87/0	<0.001
	Syntax-only struct. – Minimal	-0.4	[-4.2, +3.2]	8/9	1.000
4.1	Structured – Prose	-12.1	[-19.1, -5.7]	0/11	0.004
	Interactive – Structured	-31.9	[-44.2, -19.0]	2/31	<0.001
	Int. forced – Interactive	+17.6	[+4.5, +30.6]	21/5	0.007
	Int. core – Interactive	+17.6	[+7.2, +28.7]	17/1	<0.001
	Int. core – Structured	-14.3	[-24.4, -5.4]	2/15	0.007
	Prose – Minimal	+14.3	[+7.5, +21.8]	13/0	0.001
	Minimal – None	+27.5	[+15.2, +39.6]	27/2	<0.001
	Syntax-only struct. – Minimal	-2.2	[-6.8, +2.1]	1/3	0.625

Fig. 1 shows per-suite correctness, Table II the overall rates and Table III the pre-specified contrasts.

F1: Documentation is the dominant lever. Syntax facts alone (A$-$None) add $+15.7$, $+32.6$ and $+27.5$ pp (all Holm $p < 0.001$). Without documentation gpt-4.1-mini solves 1 of 267 tasks. Semantic and shape facts on top of syntax (B$-$A) add $+4.5$ (n.s.), $+8.2$ and $+14.3$ pp, growing with capability.

F2: With matched content, prose beats the structured contract. The primary contrast C$-$B is negative for every model: -6.0 pp [-12.5, 0.0] for nano (Holm $p = 0.083$), -7.9 pp [-12.6, -3.7] for mini ($p < 0.001$) and -12.1 pp [-19.1, -5.7] for gpt-4.1 ($p = 0.004$, 11 vs. 0 discordant tasks). The effect is concentrated on real-world tasks: on KernelBench, gpt-4.1 solves 30.8% with prose and 11.5% with the contract (-19.2 pp, $p = 0.016$), while on synthetic tasks the two are within 3 pp (89.7% vs. 87.2%), reproducing our exploratory Study 1 numbers on the same tasks (88.6% vs. 91.2%). The format of syntax facts alone does not matter (N$-$A: -3.4, -0.4, -2.2 pp, all n.s.). The penalty therefore comes from expressing semantic and shape constraints as formal predicates. For gpt-4.1 the extra structured failures are mostly shape/type verifier errors (32 vs. 13 first attempts), and for mini mostly assembly and attribute errors (62 vs. 35). This *reverses* Study 1, where an unmatched contract (schema-only syntax, 31% longer) appeared to cost the smallest model -39.5 pp and to help the largest. Unmatched documentation comparisons can mislead in both directions.

F3: On-demand access underperforms upfront delivery, for three reasons. D$-$C is -4.1 (n.s.), -13.5 and -31.9 pp. *Uptake:* gpt-4.1 never called the retrieval tool (0 of 91 tasks), replicating Study 1, while nano and mini averaged 1.3 and 2.4 calls. *Forcing helps only the model that would not ask:* requiring a first-turn call (D*) lifts gpt-4.1 by $+17.6$ pp ($p = 0.007$; Study 1 post-hoc: $+21.1$) but leaves mini and nano unchanged, and still trails C by 14 pp. *Partial documentation suppresses lookup:* giving the cross-cutting rules upfront (D$^+$) helps gpt-4.1 ($+17.6$ pp over D, $p < 0.001$), but nano then almost stops retrieving (0.02 calls per trial, down from 1.34) and loses -9.0 pp ($p < 0.001$). For every model D$^+$ remains 13–14 pp below C. Delivering the cross-cutting facts upfront, which we proposed after Study 1, is therefore not sufficient.

F4: Agents fail at well-formedness, not numerics. Across models, 99.2–99.8% of failed first attempts were rejected by the parser, the verifier or the signature check. Only 15 of the 5,000 analysed first attempts executed and produced wrong values. The failure mix is dominated by invalid assembly syntax (33–44%), shape/type verifier errors (14–27%), malformed literals (16–22%) and attribute encoding (13–16%). Up to two repairs add 0.3–14.2 pp, with the largest gain for mini under D$^+$.

V. IMPLICATIONS FOR AGENT-FACING VISAS

Validate interface claims with matched treatments. The same comparison gave opposite answers before and after we equalized content, examples and length. Agent-facing ISA documentation should be evaluated with fact-equivalent, length-matched variants. **Prefer explained constraints to bare predicates.** A machine-readable contract is useful for tools, but agents applied shape constraints better when they were stated in prose. An ISA can ship both, generated from one fact inventory, and give agents the prose. **Do not rely on agents to ask.** Strong agents skipped retrieval, and partially documented agents stopped retrieving. Introspection APIs need uptake design, such as a required first query or diagnostics that cite the fact to read, and upfront delivery of what every program needs. **Make well-formedness learnable.** Most failures are static. A normative, version-pinned surface syntax with concrete instances, plus diagnostics that name the violated rule, targets the actual bottleneck.

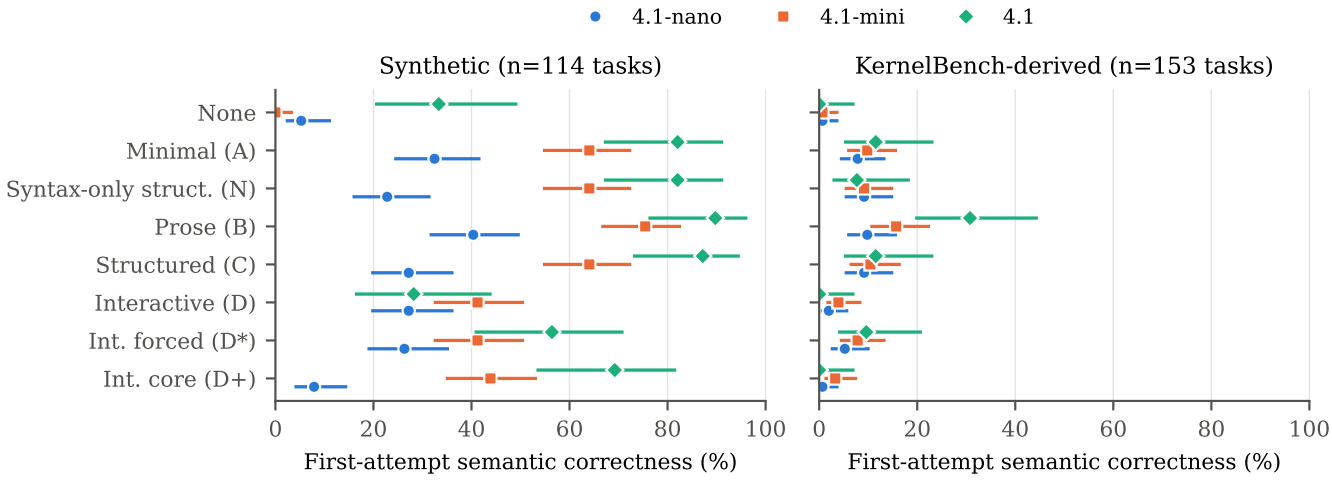


Fig. 1. First-attempt semantic correctness by interface condition and task suite (95% Wilson intervals). nano and mini: all tasks; gpt-4.1: the 39 synthetic and 52 KernelBench tasks completed within budget.

VI. Threats to Validity

Models: three snapshots of one family at temperature 0, one trial per cell. gpt-4.1 covers 91 of 267 tasks because of the budget cap, so its estimates have wider intervals. *Tasks:* synthetic tasks are small and fully specified, and KernelBench tasks were shrunk and exclude large 3D transposed convolutions. Because almost all failures are static, these results speak to syntactic and structural programmability more than to semantic reasoning. *Treatments:* the fact inventory and audit were authored by the study team with AI assistance; the audit is necessary but not sufficient, and an independent human audit is pending. Prose and contract differ in notation by design (sentences vs. predicates); our claim concerns that difference at matched content and length. *Agent loop:* repairs are harness-driven. We implemented a closed-loop agent track (self-directed verify, run-example and submit tools) but did not run it within this budget, so we make no claims about fully autonomous agents. *Procedure:* two robustness fixes during Study 2 (API key precedence, a call watchdog after network stalls) changed no prompt, grading or analysis. Both are recorded as dated addenda.

VII. Conclusion

Holding the IR, toolchain, tasks and agent fixed, the agent-facing interface of a virtual ISA changes correctness by tens of percentage points. Documentation presence dominates. At matched content, prose beats a formal contract, most strongly on real-world lowering tasks. On-demand access fails unless uptake is designed in. Agent-facing VISAs should be engineered and measured as interfaces, with matched controlled experiments.

Artifact Availability

Harness, fact inventory, task generator and KernelBench converter, oracles, frozen protocols with hashes, all trial logs and transcripts: [repository URL to be added].

References

[1] C. Lattner and V. Adve, "LLVM: A compilation framework for lifelong program analysis & transformation," in *International Symposium on Code Generation and Optimization (CGO)*, 2004, pp. 75–86.

[2] C. Lattner, M. Amini, U. Bondhugula, A. Cohen, A. Davis, J. Pienaar, R. Riddle, T. Shpeisman, N. Vasilache, and O. Zinenko, "MLIR: Scaling compiler infrastructure for domain specific computation," in *IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*, 2021, pp. 2–14.

[3] A. Ouyang, S. Guo, S. Arora, A. L. Zhang, W. Hu, C. Ré, and A. Mirhoseini, "KernelBench: Can LLMs write efficient GPU kernels?" in *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, 2025.

[4] J. Li, S. Li, Z. Gao, Q. Shi, Y. Li, Z. Wang, J. Huang, H. Wang, J. Wang, X. Han, Z. Liu, and M. Sun, "TritonBench: Benchmarking large language model capabilities for generating Triton operators," in *Findings of the Association for Computational Linguistics: ACL 2025*, 2025, pp. 23 053–23 066.

[5] Y. Guan, Y. Lin, X. Zhao, J. Yao, X. Qiang, Z. Yu, P. Viswanath, Y. Ding, and A. Aziz, "TritonGym: A benchmark for agentic LLM workflows in Triton GPU code generation," in *Proceedings of the 43rd International Conference on Machine Learning (ICML), PMLR 306*, 2026, pp. 37 457–37 473.

[6] Z. Yang, H. Ding, F. Liu, Y. Cheng, H. Cheng, Z. Chai, and H. Zhou, "CIRBench: Evaluating large language models as LLVM IR optimizers," in *Proceedings of the 43rd International Conference on Machine Learning (ICML), PMLR 306*, 2026, pp. 144 491–144 544.

[7] J. Nie, H. Wu, Y. Lai, Z. Cao, C. Zhang, B. Lou, E. Wang, J. Cheng, T. M. Jones, R. D. Mullins, R. Antonova, and Y. Zhao, "KernelCraft: Benchmarking for agentic close-to-metal kernel generation on emerging hardware," in *Proceedings of the 43rd International Conference on Machine Learning (ICML), PMLR 306*, 2026, pp. 93 070–93 121.

[8] OpenXLA, "StableHLO specification," https://github.com/openxla/stablehlo/blob/23d2f926aefe39e15a07d9874ec6871bb671b437/docs/spec.md, commit 23d2f926, accessed 2026-10-13.

[9] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press, "SWE-agent: Agent-computer interfaces enable automated software engineering," in *Advances in Neural Information Processing Systems 37 (NeurIPS)*, 2024.

[10] S. Zhou, U. Alon, F. F. Xu, Z. Wang, Z. Jiang, and G. Neubig, "DocPrompting: Generating code by retrieving the docs," in *International Conference on Learning Representations (ICLR)*, 2023.